PROJECT DOCUMENTATION

Ubuntu Terminal Online

A Browser-Based Linux Practice and Operating Systems Learning Environment

Enabling Ubuntu terminal practicals on Windows laboratory computers and other browser-capable devices

Submitted by: Faiz Shaikh
Role: Teaching Assistant
Programme: M.Tech - First Year
Submission: College Project Report
Academic Year: 2026-27

Contents

The report is organised around the educational problem, the implemented solution, the verified features, and the observed results.

Abstract

Many computers in the college laboratory run Microsoft Windows and do not provide a native Ubuntu installation. A few systems may also be unavailable or have configuration issues, which reduces the number of machines on which students can complete Linux, shell scripting, C programming, and Operating Systems practicals.

To address this constraint, Faiz Shaikh - Teaching Assistant and first-year M.Tech student at the same institution - developed Ubuntu Terminal Online. The website reproduces the familiar Ubuntu GNOME Terminal experience in a browser and provides an educational shell, cached filesystem, C practical workflow, process simulation, preloaded laboratory programs, and interactive concept visualisations. Students can therefore practise from Windows computers or other modern browser-capable devices without changing the host operating system.

> **PROJECT OUTCOME** A single browser-based learning surface now supports routine Ubuntu terminal practice, C and shell exercises, and visual Operating Systems demonstrations while preserving each student's working files in that browser.

1. Problem Statement

Linux-based laboratory exercises assume that every learner has reliable access to Ubuntu. In practice, many laboratory computers contain Windows, some systems may have installation or maintenance issues, and dual boot or virtual-machine setup can consume class time. Students may also want to practise from personal devices where they cannot install another operating system.

The academic need was therefore not just a terminal-looking interface. The required solution had to let students enter realistic Linux commands, create and edit files, run shell scripts, compile classroom C programs, observe process behaviour, and return to their work after a page reload.

2. Project Objectives

- Provide an Ubuntu-like terminal that runs inside a web browser.
- Support the commands and scripting constructs required for common Linux and Operating Systems practicals.
- Allow students to create, edit, compile, and execute files without requiring Ubuntu on the host computer.
- Persist student files in browser storage while also allowing a clean reset for the next laboratory session.
- Explain difficult OS concepts through step-by-step visual lessons in addition to command-line practice.

3. Proposed Solution

Ubuntu Terminal Online is a Next.js web application that combines a client-side Bash-style interpreter with a virtual Linux filesystem and a classroom-focused C execution engine. It presents the prompt, colours, title bar, keyboard workflow, history, and editor interactions expected from an Ubuntu terminal. For tools that require a full operating system, the command ubuntu-vm opens a full WebVM-based Ubuntu environment inside the application.

4. System Architecture

1. Presentation layer: React components render the Ubuntu-styled terminal, split view, editor, reset controls, and visual lessons.
2. Command layer: the shell parser interprets commands, variables, control structures, pipelines, redirection, command substitution, arithmetic, and executable files.
3. Execution and storage layer: built-in commands operate on a virtual filesystem saved in browser localStorage; compiled classroom C programs run through the educational C interpreter and simulated process table.
4. Content layer: a server route reads teacher-provided UTF-8 practical files from public/terminal-files and merges them into /home/student for learners.
5. Extension path: ubuntu-vm embeds webvm.io when a student needs apt, Python, networking, or a complete Linux userland.

5. Technology Stack

Layer	Technology	Purpose
Framework	Next.js 16.3, React 19.2	Routing, server endpoint, rendering, and application structure
Interface	JavaScript/JSX, CSS, Tailwind CSS 4	Responsive Ubuntu-style terminal and visual learning pages
Local engine	Custom shell, VFS, Mini-C, process table	Offline classroom commands, files, compilation, and OS simulation
Persistence	Browser localStorage	Retains the student's virtual home directory across reloads
Full Linux option	WebVM embedded experience	Access to a broader Ubuntu environment when internet access is available

6. Key Features

- **Ubuntu-like terminal interface.** GNOME-terminal-inspired colours, title bar, welcome banner, prompt, command history, keyboard input, clear/exit behaviour, and responsive layout.
- **Practical shell environment.** Supports variables, aliases, environment variables, if/case conditions, for/while/until loops, functions, tests, pipelines, logical operators, globs, redirection, command substitution, and arithmetic expansion.
- **Virtual Linux filesystem.** Includes familiar directories, permissions, timestamps, file operations, text processing tools, named FIFOs, and a home directory at /home/student.
- **Browser-cached work.** Files persist in localStorage across page reloads. A daily reset returns shared laboratory computers to a clean state, while Hard Reset and factory-reset provide manual recovery.
- **C practical support.** Students can edit C source, compile with gcc/cc/g++, create an executable, accept input with scanf, and use classroom functions such as printf, fork, wait, getpid, pipes, files, and System V message queues.
- **Shared split terminal.** Two side-by-side terminals use the same virtual filesystem, allowing one pane to create or compile a file and the other to inspect or execute it.
- **Integrated editor.** nano, vi, vim, and gedit commands open an in-browser editor for the selected virtual file.
- **Preloaded practicals.** Teacher-managed programs are supplied through the terminal-files endpoint. The current repository includes exercises for practicals 6, 7, and 8 plus reader/writer and send/receive examples.
- **Interactive OS Visual Lab.** Dedicated lessons demonstrate program-to-process transition, fork(), waiting, zombie and orphan states, and IPC through named pipes, message queues, shared memory, and signals.
- **Full Ubuntu escape hatch.** The ubuntu-vm command opens a WebVM session for tasks that need apt, python3, networking, or a broader toolchain than the cached classroom shell.

> **ACCESSIBILITY ADVANTAGE** Because the main experience runs in a browser, the same lesson can be opened from a Windows lab PC, laptop, or other modern device without repartitioning, dual boot, or a local Ubuntu installation.

7. Implementation Details

7.1 Terminal and Session Management

UbuntuTerminal.jsx manages the prompt, cursor, command history, output blocks, input mode, editor state, split-terminal labels, and WebVM overlay. A session object keeps the current working directory, shell variables, aliases, history, and references to the shared virtual filesystem.

7.2 Shell Parser and Interpreter

interpreter.js tokenises and parses Bash-like input into executable structures. It expands variables and globs, evaluates arithmetic and command substitution, executes conditional and loop nodes, connects pipeline stages, and applies input/output/error redirection. Built-ins are dispatched from commands.js, while virtual executable files are handled by the execution layer.

7.3 Virtual Filesystem and Persistence

fs.js models directories, regular files, FIFOs, permissions, modification times, copy/move/remove operations, and path normalisation. The tree is serialised to localStorage after changes. Files distributed by the teaching staff are read by /api/terminal-files and merged into the learner's home directory.

7.4 C and Process Simulation

minic.js implements the subset of C required by the supplied practicals, including standard input/output, arrays, loops, functions, file APIs, process calls, signals, pipes, and System V message queues. process-table.js tracks PID, PPID, running/sleeping/zombie state, parent-child relationships, reaping, and orphan adoption by PID 1.

7.5 Visual Concept Lessons

ProcessConcept.jsx defines progressive lessons with labelled stages and selectable scenarios. Students can move one step at a time or play the animation, connecting source-code concepts with memory, queues, CPU scheduling, process tables, system reapers, and IPC objects.

> **DESIGN CHOICE** Routine practicals remain fast and local in the cached educational shell; advanced commands are deliberately delegated to a full WebVM environment. This hybrid approach balances classroom reliability with access to a broader Ubuntu userland.

8. How to Use the Website

6. Open the website in a modern browser on the laboratory or personal device.
7. Click inside the terminal and type help to view the available classroom commands.
8. Use pwd, ls, cd, mkdir, cat, grep, chmod, and related commands exactly as in a Linux practical.
9. Create or edit scripts and source files using nano filename, vi filename, vim filename, or gedit filename.
10. Compile a C program using gcc program.c -o program and execute it with ./program.
11. Choose Split Terminal when two coordinated shells are useful; both panes immediately see the same files.
12. Use the OS Visual Lab routes for process, fork, zombie/orphan, and IPC demonstrations.
13. Run ubuntu-vm when a task needs apt, python3, networking, or the full Ubuntu toolchain.
14. Use Hard Reset only when cached files or running simulations must be cleared; shared lab data also resets automatically on a new browser day.

8.1 Example Practical Workflow

```
$ pwd
/home/student
$ ls
hello.c  hello.sh  practicals  ...
$ gcc hello.c -o hello
$ ./hello
Hello, Ubuntu!
```

The screenshots on the following pages were captured from the production build of this repository on 2 September 2026.

9. Website Screenshots and Feature Demonstration

Figure 1 shows a complete terminal workflow: location check, directory listing, C compilation, and execution. The visible Hello, Ubuntu! output verifies that the generated executable runs inside the classroom environment.

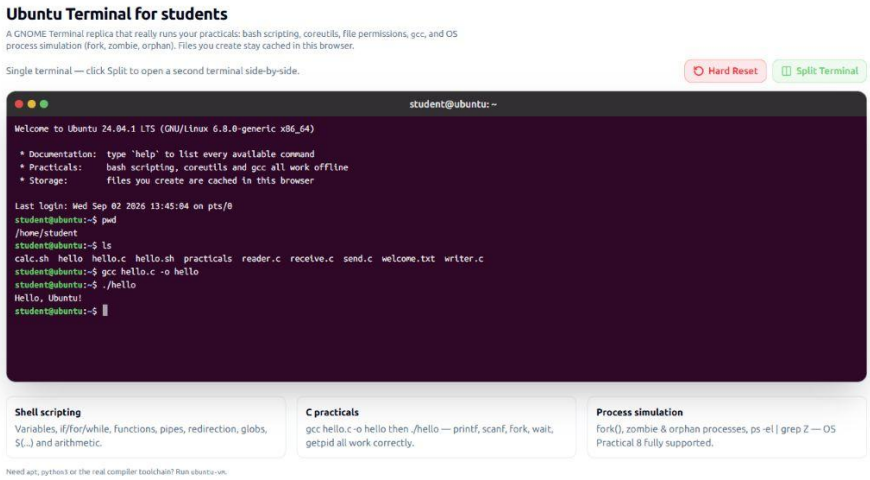

Figure 1. Ubuntu-style terminal with filesystem access and C compilation.

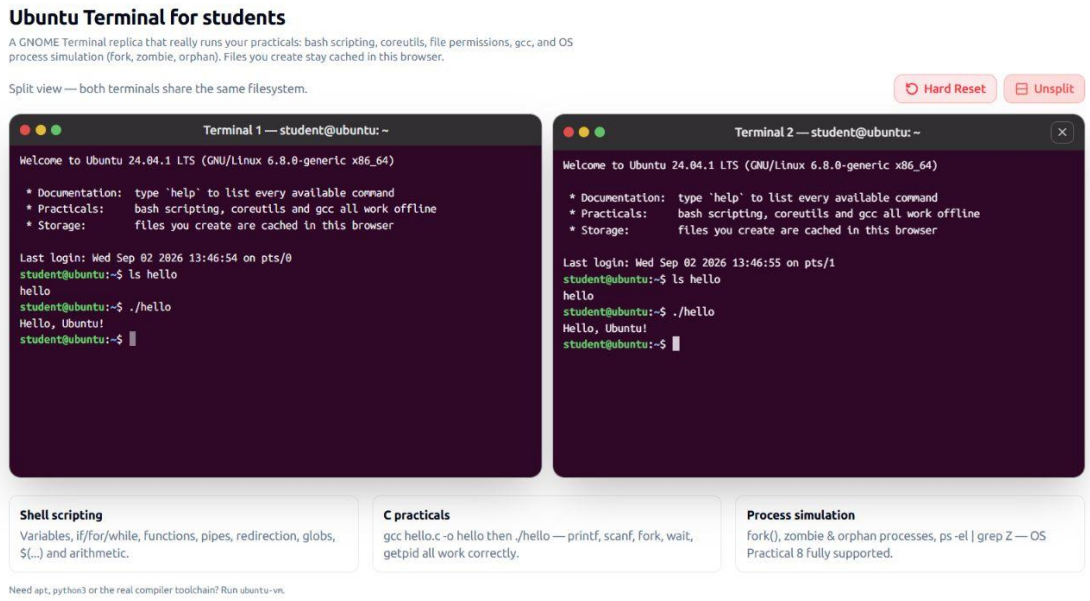

Figure 2. Split-terminal mode; both panes can access the shared executable and filesystem.

9.1 Interactive Operating Systems Visualisations

The visual lab complements command execution with staged explanations. Learners can select a stage directly, move with Next, or play the sequence as an animation.

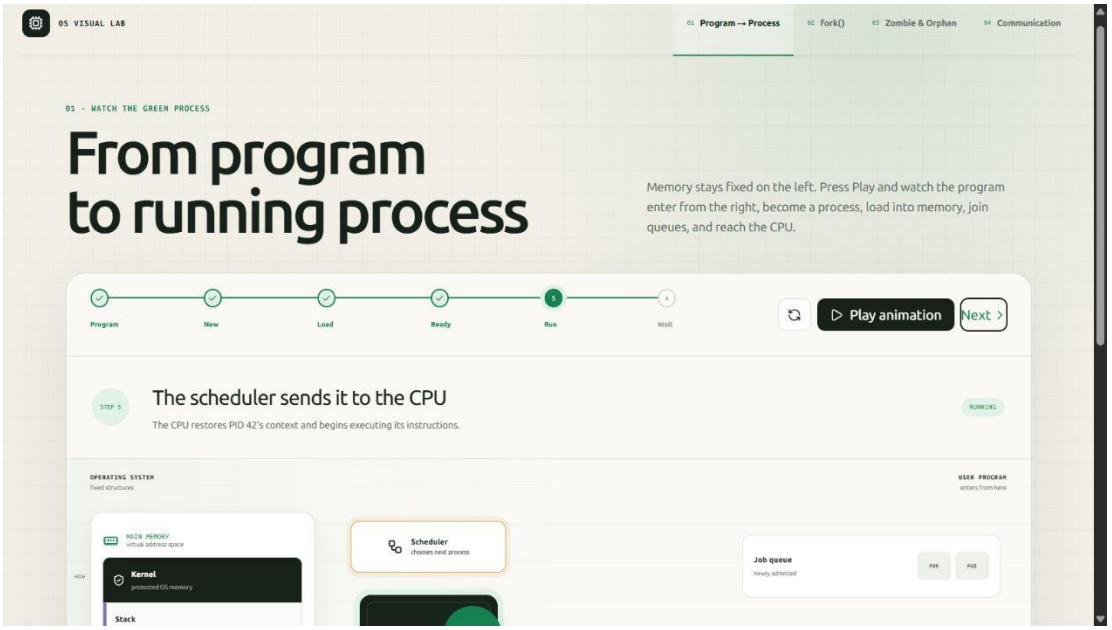

Figure 3. Program-to-process lesson at the scheduling and CPU execution stage.

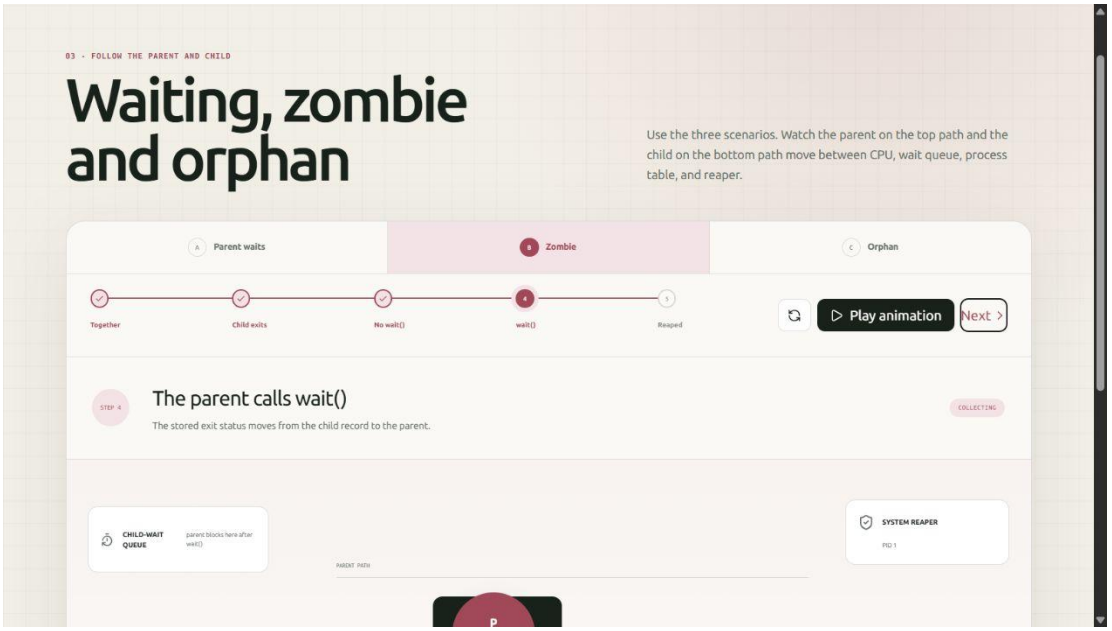

Figure 4. Zombie-process scenario showing wait(), the process table, and reaping.

9.2 Inter-Process Communication

The communication lesson presents named pipes, message queues, shared memory, and signals using a consistent five-stage sequence: separate processes, create IPC, send, transfer, and receive. The selected screenshot shows a message travelling through a kernel-managed queue while the two process address spaces remain separate.

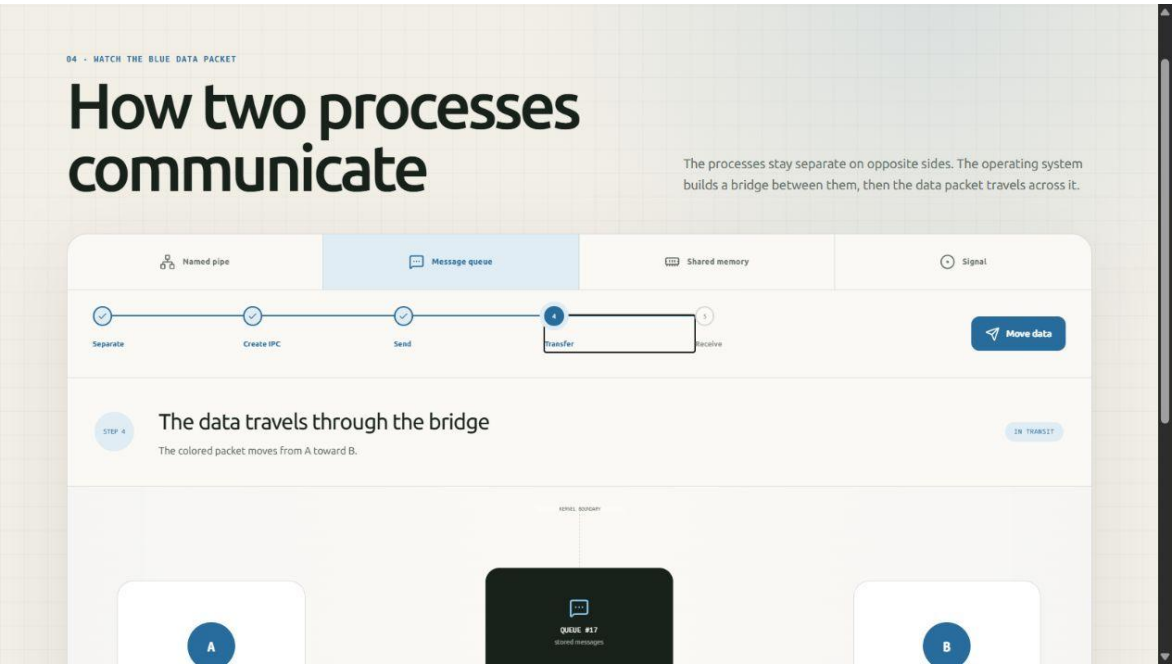

Figure 5. System V-style message queue visualisation during data transfer.

> **TEACHING VALUE** The same concepts can first be observed visually and then practised through C programs that use fork(), wait(), pipes, files, or message-queue functions.

10. Testing and Results

The project was verified using an optimised production build and representative classroom tasks. The build completed successfully and generated routes for the terminal, file API, and all four visual lessons.

Test	Procedure	Observed Result
Production build	npm run build	Compiled successfully; application routes generated
Navigation	pwd and ls	Displayed /home/student and the expected cached/preloaded files
C workflow	gcc hello.c -o hello; ./hello	Program executed and printed Hello, Ubuntu!
Shared storage	Run the same executable in both split panes	Both terminals accessed the same file and output
Content delivery	Load /api/terminal-files	Practical source files appeared under /home/student
Process lesson	Advance to the Run stage	Scheduler, CPU, memory, and queue state updated visually
Zombie lesson	Select Zombie and advance to wait()	Exit status and reaping relationship were displayed
IPC lesson	Select Message queue and Transfer	Message packet moved through the queue from A toward B

RESULT The implemented features satisfy the core requirement: students can complete common Ubuntu-oriented practical work on a Windows laboratory computer through the browser, while advanced Linux operations remain available through the full-VM option.

10.1 Quality and Safety Characteristics

- Routine commands execute locally in the browser and do not run arbitrary programs on the application server.
- Student-created files remain in that browser's storage unless reset; the design avoids a shared server-side student filesystem.
- Teacher-provided practical files are limited to readable UTF-8 text content.
- Hard Reset provides a visible recovery path for corrupted files or stuck simulations on shared machines.

11. Benefits and Educational Impact

- **Higher access.** Students are no longer limited to the subset of laboratory systems with a working Ubuntu installation.
- **Faster class start.** No dual-boot configuration or per-student virtual-machine setup is needed for routine exercises.
- **Practice continuity.** Browser-cached files survive refreshes, enabling learners to continue code and shell work during the same period.
- **Better conceptual understanding.** Animations connect commands and C system calls to CPU scheduling, process states, wait semantics, and IPC.
- **Simpler teaching distribution.** New or corrected practical files can be placed in the public terminal-files folder and delivered to learners on page load.

12. Limitations and Future Scope

The cached shell is an educational implementation rather than a complete Linux kernel and GNU userland. Some commands implement the options needed for classroom use rather than every option available on Ubuntu. Browser localStorage has a capacity limit, binary files are not distributed through the practical-file endpoint, and the full WebVM option depends on internet access and the availability of webvm.io.

Future work can add authenticated student workspaces, instructor dashboards, cloud backup, assignment submission, automated output checking, more command options, additional C library functions, mobile-specific controls, accessibility testing, offline packaging, and institution-hosted full Linux containers or WebAssembly sandboxes.

13. Conclusion

Ubuntu Terminal Online directly addresses a real college laboratory constraint: many computers run Windows and some systems cannot reliably provide Ubuntu. The project converts any suitable browser into a practical Linux learning surface with a familiar terminal, persistent files, shell scripting, C compilation, process simulation, supplied laboratory programs, split-terminal collaboration, and interactive OS explanations.

Developed by Faiz Shaikh in his combined role as Teaching Assistant and first-year M.Tech student, the website is both an infrastructure solution and a teaching aid. It reduces dependence on machine configuration while giving students more opportunities to practise, experiment, and understand operating-system behaviour.

Appendix A - Supported Command Groups

The following groups summarise the commands implemented in the cached classroom shell. Availability of every GNU option is not implied; the implementation focuses on the options used in laboratory exercises.

Group	Examples
Navigation and files	pwd, cd, ls, tree, mkdir, rmdir, rm, touch, cp, mv, ln, stat, file, find, du, df
Text processing	cat, head, tail, wc, grep, sort, uniq, cut, tr, sed, awk, tee
Permissions and FIFOs	chmod, mkfifo
Shell and scripting	echo, printf, read, test, [, [[, export, env, alias, source, bash, sh, loops, functions, case
System information	date, cal, whoami, id, hostname, uname, uptime, free, ps, top
Editors	nano, vi, vim, gedit
Toolchain	gcc, cc, g++, make
Session and recovery	history, which, man, clear, reset, factory-reset, exit, logout
Full environment	ubuntu-vm for apt, apt-get, python3, networking, and broader Ubuntu tools

Appendix B - Main Project Modules

Module	Responsibility
app/page.jsx	Main terminal workspace, split view, reset actions, and feature cards
src/components/UbuntuTerminal.jsx	Interactive terminal user interface and session integration
src/lib/shell/interpreter.js	Bash-like parsing, expansion, control flow, pipelines, and redirection
src/lib/shell/commands.js	Built-in Linux commands, editors, compiler commands, and VM launch
src/lib/shell/fs.js	Virtual filesystem, permissions, persistence, daily reset, and public-file merge
src/lib/shell/minic.js	Classroom C parsing/execution and operating-system APIs
src/lib/shell/process-table.js	PID/PPID state, zombie reaping, and orphan adoption
src/components/process-concepts/	Interactive process, fork, zombie/orphan, and IPC lessons

Prepared from the verified project repository and production screenshots - 2 September 2026